

Week- 4

# Backend Engineering

Pawan Dhanjwari

# Launchpad

# **Backend Development Rest API and Java Frameworks for Backend Development**

# Agenda

- What is API and introduction to Rest API
- REST API and HTTP verbs
- Modelling restful endpoints
- Frameworks for java web applications
- Live coding
- Question and answers

“An API is a precise specification of the programming instruction and standards to access a web based software or web tool...”

# What is an API?

- ▶ Acronym for Application program interface
- ▶ Acts as a bridge between the programmer and application
- ▶ Takes specific requests predefined when created
- ▶ Verifies the request and processes the data
- ▶ Performs the processing and returns the response to the user

# What is REST

- ▶ REST stands for representational state transfer
- ▶ REST is not a framework but an architectural principle
- ▶ REST uses http which is oriented around verbs and resources (GET, POST, PUT, DELETE)

# What is REST (continued...)

- Verbs are then applied to resources
- Request consists of headers, body and methods
- Response consists of headers, status code and body
- Data represented via XML/JSON/HTML

# Key Principles of REST

- ▶ Stateless
- ▶ Client Server Architecture
- ▶ Uniform Interfaces
- ▶ Resource Oriented

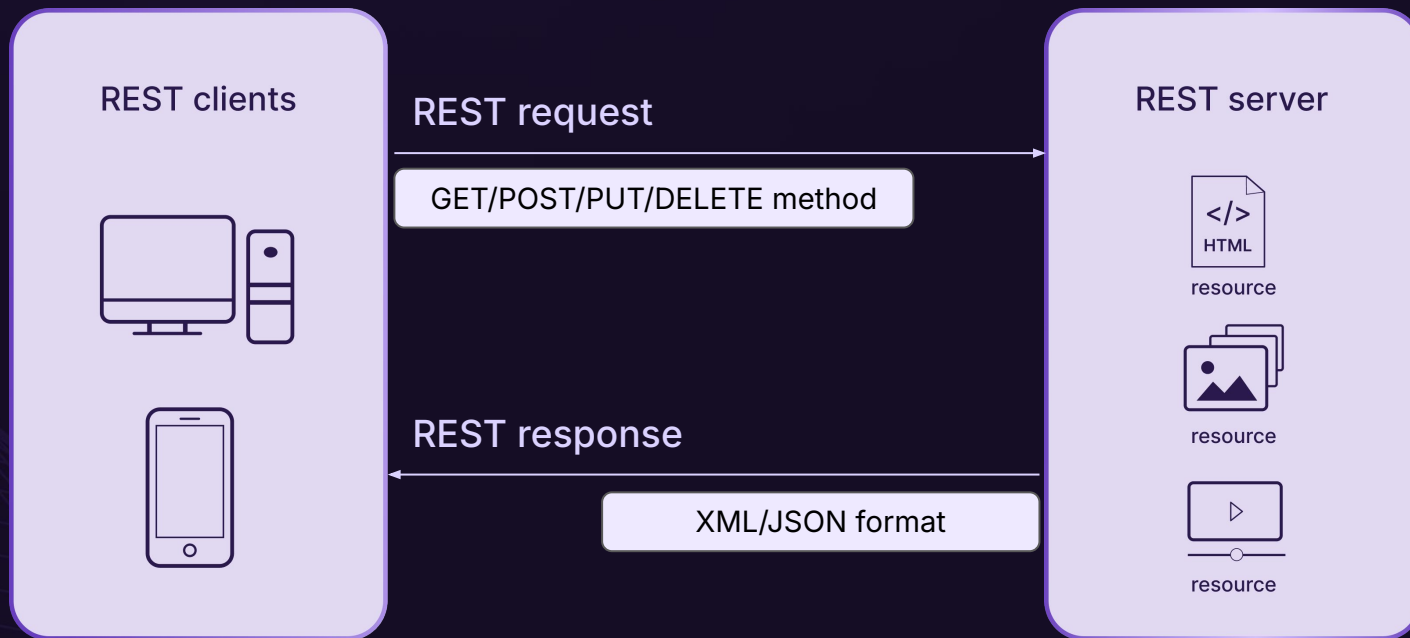


# API architectural styles

|                       | REST                                          | SOAP                                                                                                                 | RPC                                                                                                            | GraphQL                                      |
|-----------------------|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|----------------------------------------------|
| Organized in terms of | compliance with six architectural constraints | enveloped message structure                                                                                          | local procedure calling                                                                                        | schema & type system                         |
| Format                | XML, JSON, HTML, plain text,                  | XML only                                                                                                             | JSON, XML, Protobuf, Thrift, FlatBuffers                                                                       | JSON                                         |
| Learning curve        | Easy                                          | Difficult                                                                                                            | Easy                                                                                                           | Medium                                       |
| Community             | Large                                         | Small                                                                                                                | Large                                                                                                          | Growing                                      |
| Use cases             | Public APIs simple resource-driven apps       | Payment gateways, identity management<br>CRM solutions financial & telecommunication services, legacy system support | Command and action-oriented APIs;<br>internal high performance communication in massive micro-services systems | Mobile APIs, complex systems, micro-services |

# Rest API architecture

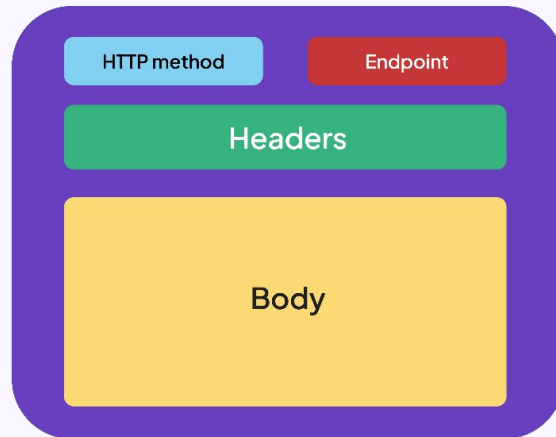
## REST API IN ACTION



# REST API request example

```
POST /api/2.2/sites/9a8b7c6d-54f-3a2b-1c0d-98f7a6b5c4d/users HTTP/1.1
HOST: my-server
X-Tableau-Auth: 12ab34cd56ef78ab90cd12ef34ab56cd
Content-Type: application/json
```

```
{
  "user": {
    "name": "NewUser1",
    "siteRole": "Publisher"
  }
}
```



# REST API response example

```
HTTP/1.1 200 OK (285ms)
Date: Fri, 21 Apr 2017 10:27:20 GMT
Server: Apache/2.4.6 (CentOS) OpenSSL/1.0.1e-fips PHP/7.0.16
X-Powered-By: PHP/7.0.16
Content-Length: 109
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: application/json; charset=UTF-8
```

---

```
{"status": "success", "message": "City  
List", "data": [{"city_name": "Visakhapatnam"}, {"city_name": "Vijayawada"}]}
```

# HTTP response example

## HTTP Status Codes

Level 200 (Success)

OK

Created

203 : Non-Authoritative  
Information

204 : No Content

Level 400

400: Bad Request

401: Unauthorized

403: Forbidden

404: Not Found

409: Conflict

Level 500

500: Internal Server Error

503: Service Unavailable

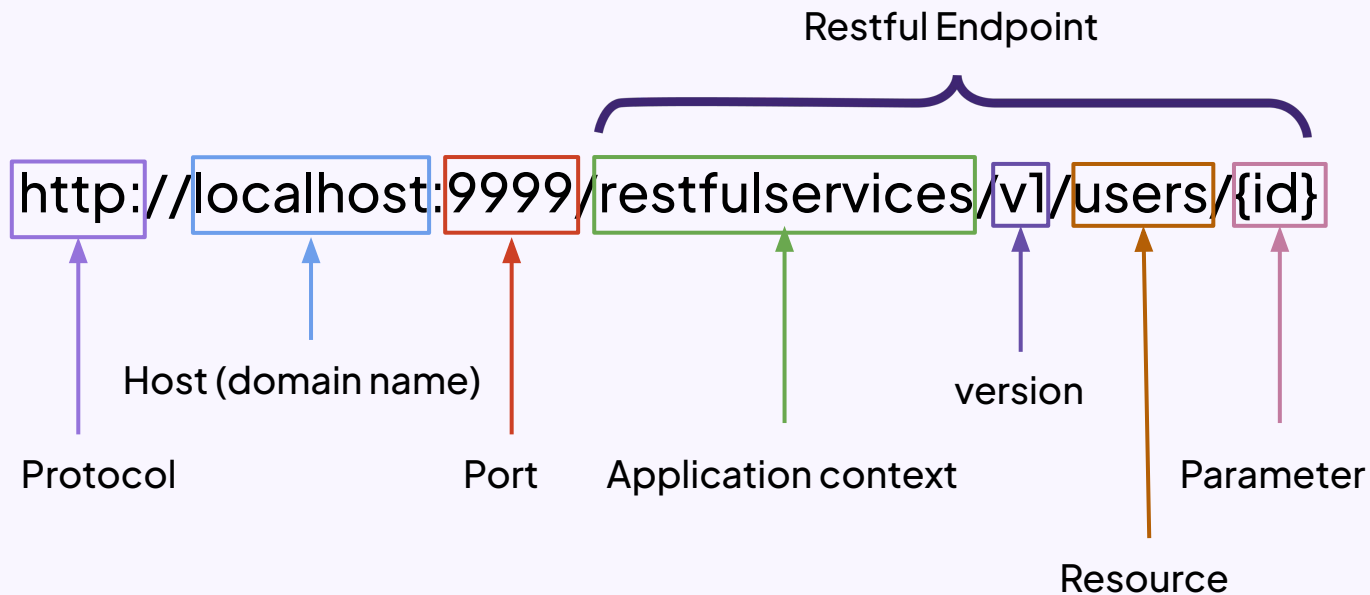
501: Not Implemented

504: Gateway Timeout

599: Network timeout

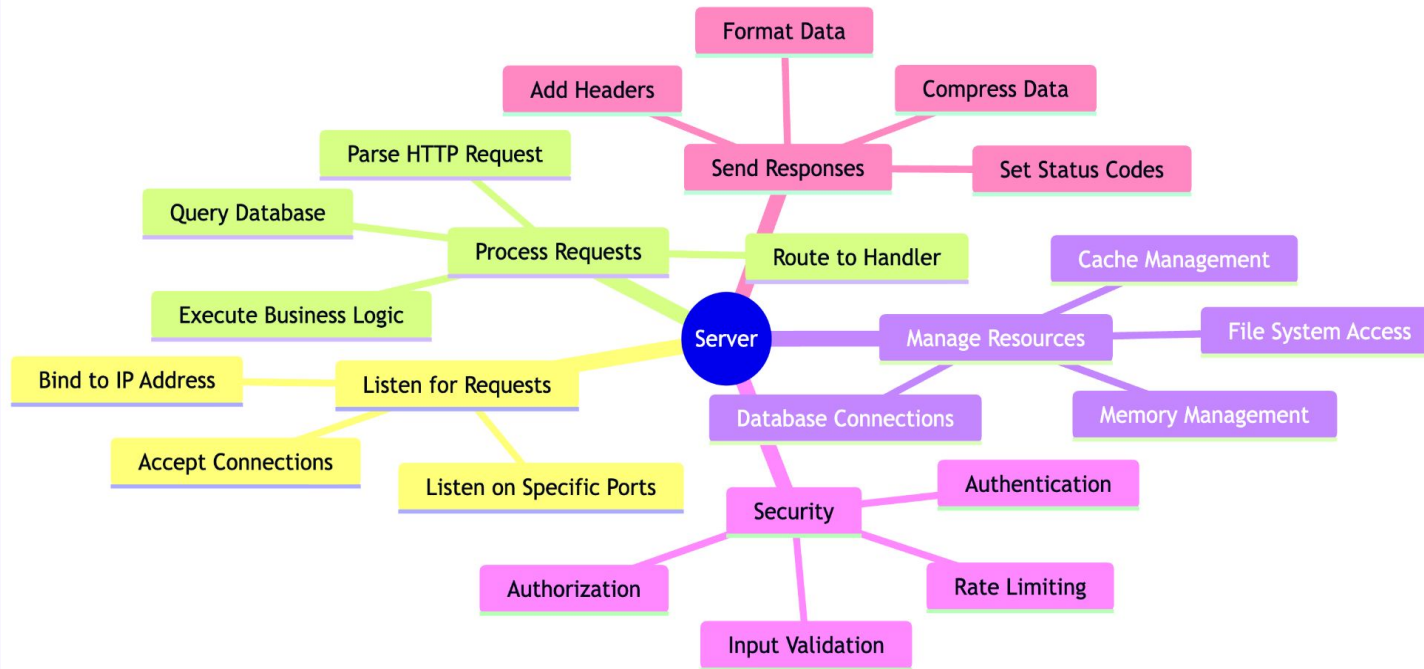
502: Bad Gateway

# Restful endpoint structure



# Server Handling

# Server Responsibilities





# From a Java Class File to a Running Web App

Each box is a stage. The final stage is what we call a 'web application'.



## Key idea:

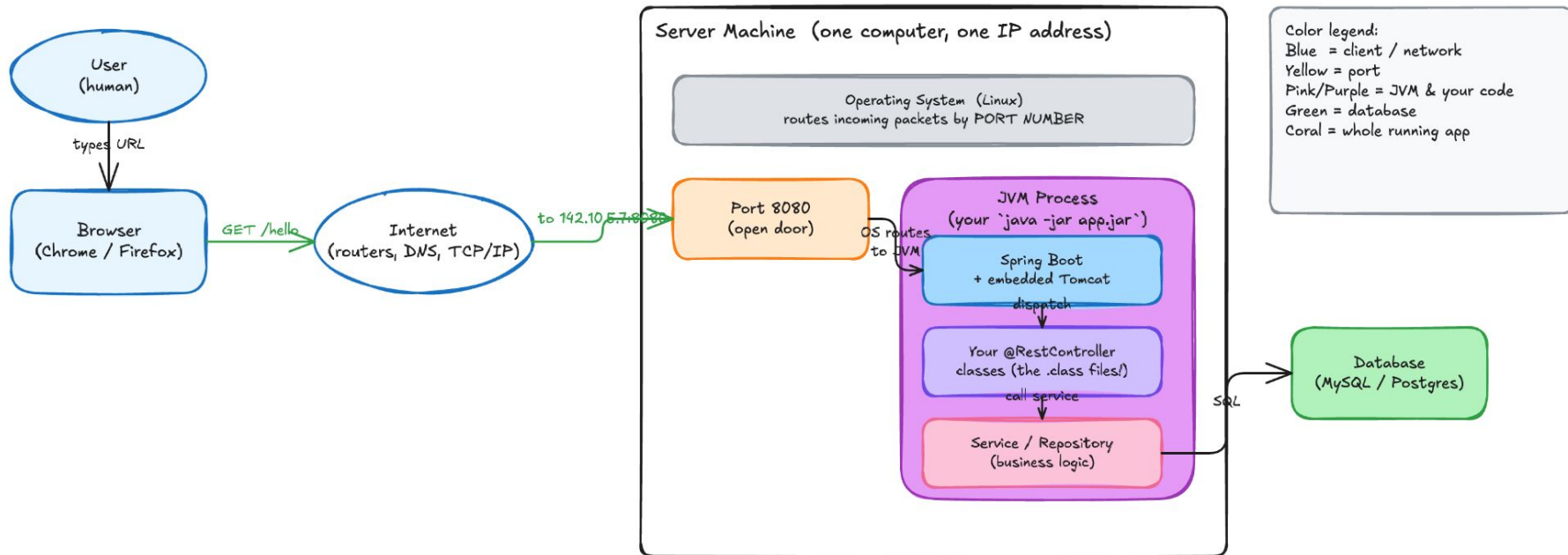
Until you run `java -jar`, your code is just files on disk.  
Once the JVM starts and Spring Boot binds to a port, that process IS the web server — a long-running program waiting at a port for HTTP requests.

What it looks like once it is running:



# What IS a Web Application?

A web application is a program that runs forever, waiting at a port, and replies to HTTP requests. Here's its anatomy on a server:

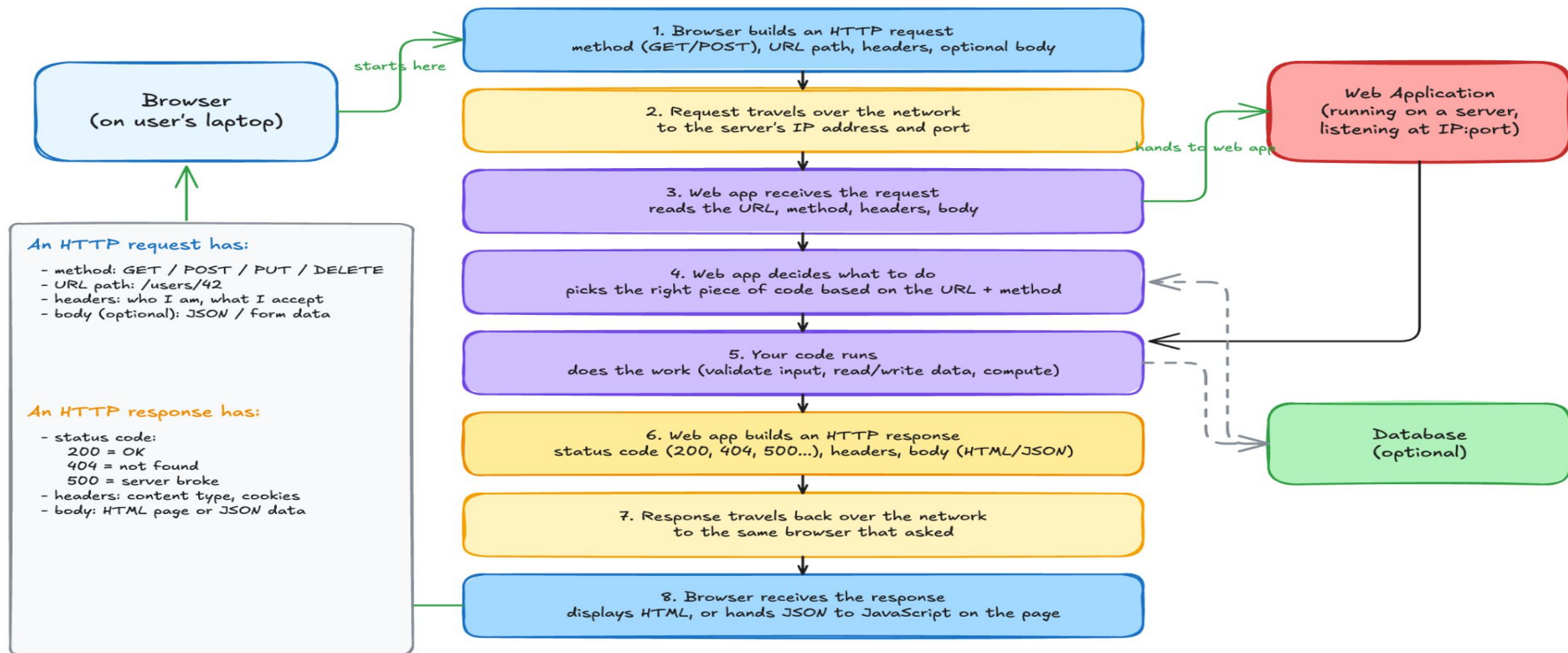


Crucial: the 'web app' is just your .class files running inside a JVM, behind a port that the OS forwards traffic to. Nothing magical.

# A Request's Lifecycle

To move canvas, hold **Scroll wheel** or **Space** while dragging, or use the hand tool

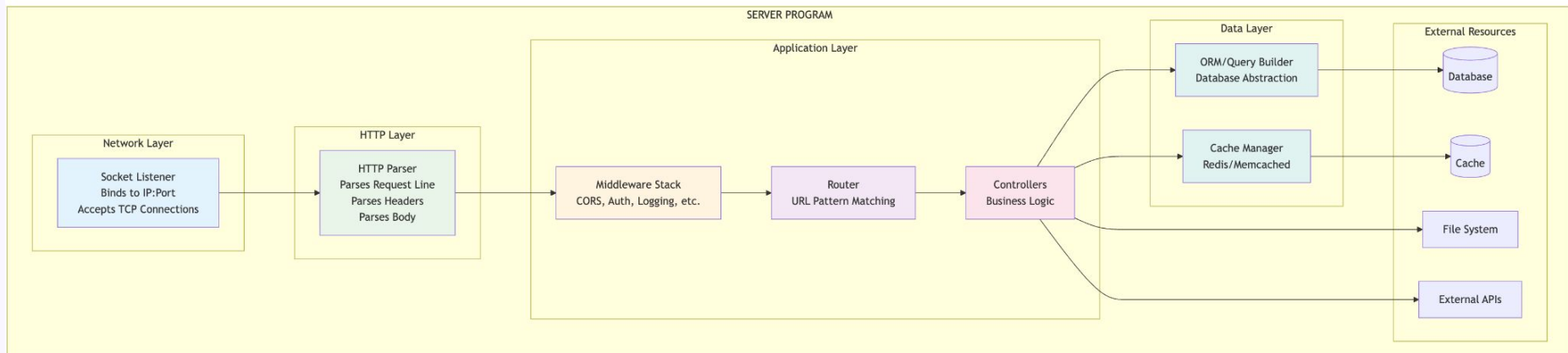
What happens, in order, every time a user clicks a link or hits a button.



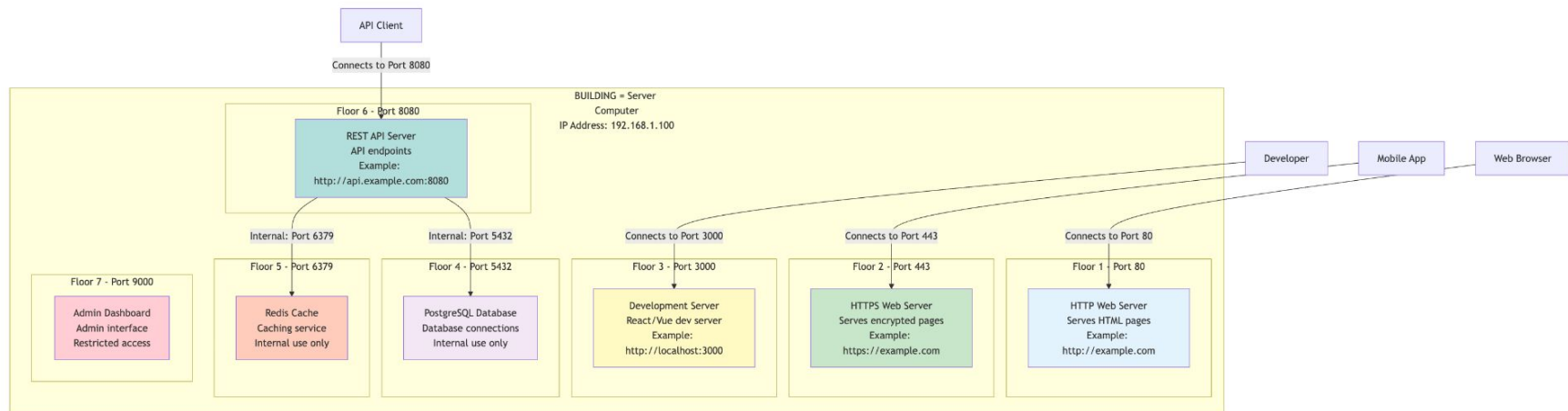
## Mental model:

A web app is a program that: (a) waits for HTTP requests, (b) decides what each request means, (c) does the work, (d) sends back an HTTP response. Every request is one trip through these eight steps. Same shape for a login click, a search query, a Like button — only the details inside change.

# Server Components



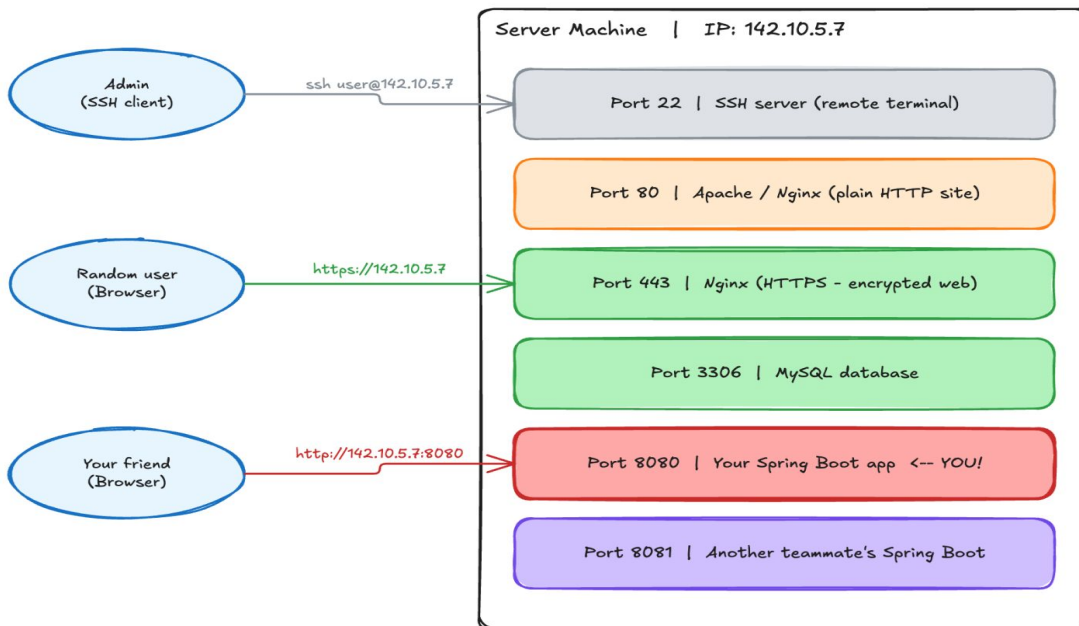
# Port Analogy



# Ports: the 'apartment numbers' of a server

A server has *ONE* IP address but can run *MANY* apps.

Each app picks a port. The OS routes each packet to the right app by its port number.



Common ports to know:

|       |                     |
|-------|---------------------|
| 22    | SSH (terminal)      |
| 80    | HTTP                |
| 443   | HTTPS               |
| 3306  | MySQL               |
| 5432  | PostgreSQL          |
| 6379  | Redis               |
| 8080  | Spring Boot default |
| 27017 | MongoDB             |

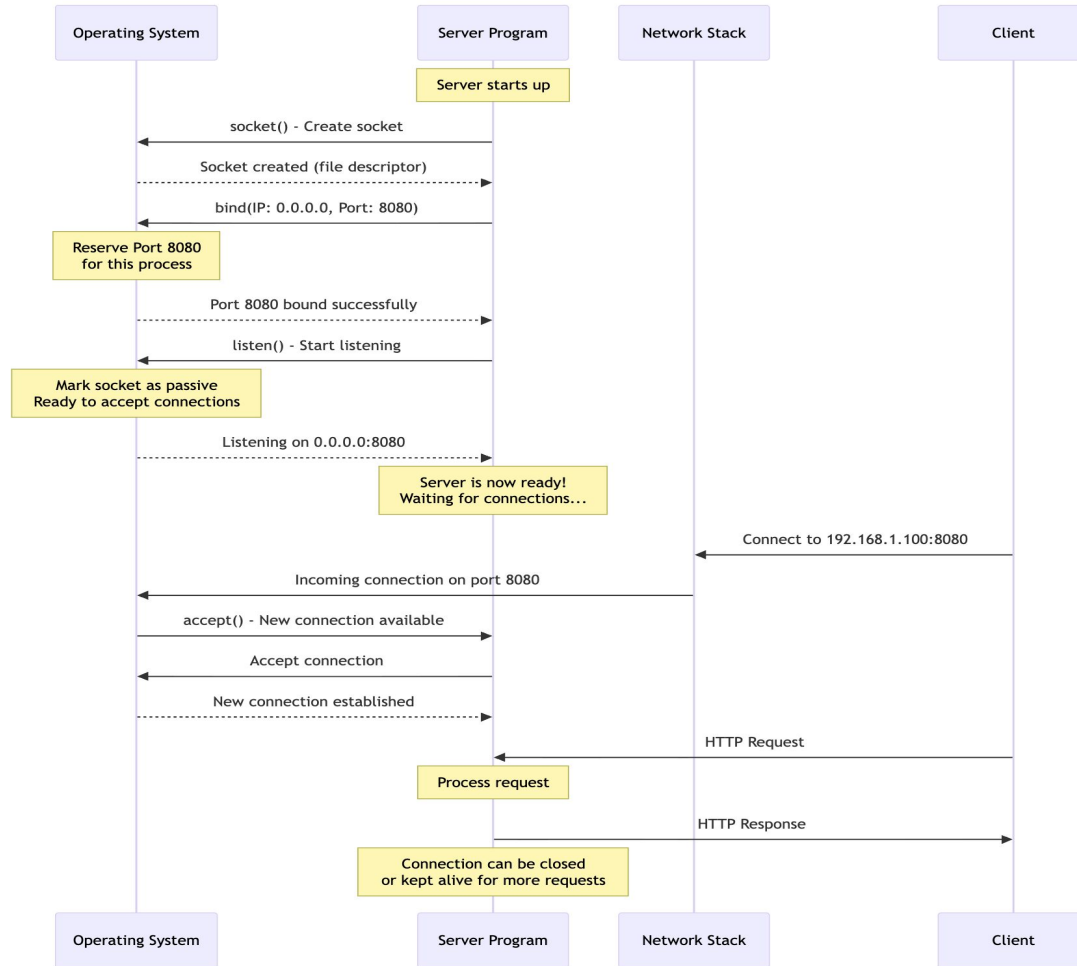
## Analogy:

IP address = the street address of an apartment building (which machine on the internet)

Port number = the apartment number inside that building (which program on that machine)

Two apps cannot share the same port at the same time — like two families can't share one apartment.

# How Ports Work



# Client Server Architecture

[Reference](#)



# Complete Request Lifecycle

[Reference](#)

# Frameworks for java web applications

# What does a Web Framework actually do for you?

A web app needs ALL the things in the middle layer.

Without a framework you write every one yourself. With Spring Boot, you get them for free.

HTTP request  
arrives



## YOUR CODE (this is all you write)

Controllers  
@GetMapping("/hello") -> a method

Business logic  
the rules unique to YOUR app

Data models  
User, Order, Product classes

## FRAMEWORK (Spring Boot does all of this for you)

Parse HTTP  
raw bytes -> request object

URL routing  
match /hello to your method

Read request body  
JSON / form -> Java object

Write response  
Java object -> JSON + headers

Many users at once  
threading & connection handling

Database access  
connection pool + JdbcTemplate/JPA

Transactions  
begin / commit / rollback

Validation  
@Valid + annotations on fields

Auth / security  
sessions, tokens, password hashing

Error handling  
exception -> 4xx/5xx status

Configuration  
application.properties, env vars

Logging & metrics  
request logs, health endpoints

## JVM + OPERATING SYSTEM (the foundation - you don't touch this either)

TCP sockets | threads & memory | file system | network drivers

## The big idea

Every web app on Earth needs the 12 chores in the middle box.

Without a framework, you would write all of them yourself — thousands of lines before you even start your business logic.

A framework is a pre-built middle layer that handles those chores for you.

Spring Boot is one such middle layer for Java.

You bring the top layer (your rules); it brings the rest.

## One-line takeaway:

A framework lets you write only the parts that are **UNIQUE** to your app, and reuses everything that is **COMMON** to all web apps.

# What is a framework ?

► **Framework:** A framework in Java is a pre-defined structure or platform that provides a foundation and standard set of functionalities to build applications. It includes libraries, tools, and best practices.

► **Characteristics:**

- Reusable code
- Inversion of control
- Standardization
- Extensible
- Integration

# Spring Framework

- What is spring framework
- Features of spring
- Lifecycle of a bean
- Dependency Injection (DI) and Inversion of control (IOC)

# Why do we need spring ?

## ► Simplification of Development

- **Boilerplate Code Reduction:** Minimizes repetitive code with features like Dependency Injection and AOP.
- **Configuration Management:** Simplifies application configuration with annotations and XML.

## ► Flexibility and Modularity

- **Modular Design:** Use only the needed components (e.g., Spring Core, Spring MVC, Spring Data).
- **Integration:** Seamlessly integrates with other frameworks and libraries (e.g., Hibernate, JMS).

# Why do we need spring ?

## ► Enterprise-Grade Features

- **Transaction Management:** Simplifies transaction management across various resources.
- **Security:** Comprehensive security framework for authentication and authorization.
- **Batch Processing:** Robust support for batch processing and task scheduling.

# Why do we need spring ?

## ► Community and Ecosystem

- **Large Community:** Extensive documentation, community support, and third-party extensions.
- **Rich Ecosystem:** Includes Spring Boot, Spring Cloud, Spring Data, and more for various enterprise needs.

## ► Rapid Development

- **Spring Boot:** Simplifies setup and accelerates development with auto-configuration, embedded servers, and opinionated defaults.



# Features of Spring

## ► Core Features

- **Dependency Injection (DI):** Allows objects to define their dependencies and inject them at runtime, promoting loose coupling and easier testing.
- **Inversion of Control (IoC):** The control of object creation and management is transferred from the application to the Spring container.

# Core components of Spring Framework

- ▶ **Spring Core** - Dependency Injection (DI) and Inversion of Control (IoC) containers.
- ▶ **Spring MVC**: Model-View-Controller framework for building web applications.
- ▶ **Spring Boot**: Simplifies the setup and development of new Spring applications with convention-over-configuration.

# Core components of Spring Framework

- ▶ **Spring Data JPA:** Abstraction over JPA for simplifying database interactions.
- ▶ **Spring Security:** For securing web applications.
- ▶ **Spring Boot:** A project that simplifies the setup and development of new Spring applications with convention-over-configuration.
- ▶ **Spring Cloud:** For building microservices and cloud-native applications.

# What is a bean ?

- ▶ **Definition:** A bean is an object that is instantiated, assembled, and managed by the Spring IoC container.
- ▶ **Lifecycle:** The container is responsible for managing the lifecycle of beans, including their creation, initialization, and destruction.

# Bean configuration methods

## ► XML Configuration

- Typically done using file `applicationContext.xml` or `spring.xml`.

## ► Java Configuration

- Typically done using a class for config annotated with `@Configuration`.

## ► Annotation configuration

- Done using annotations like `@Component`, `@Service`, `@Repository`, and `@Controller` to declare beans.

# Dependency Injection and Bean scopes

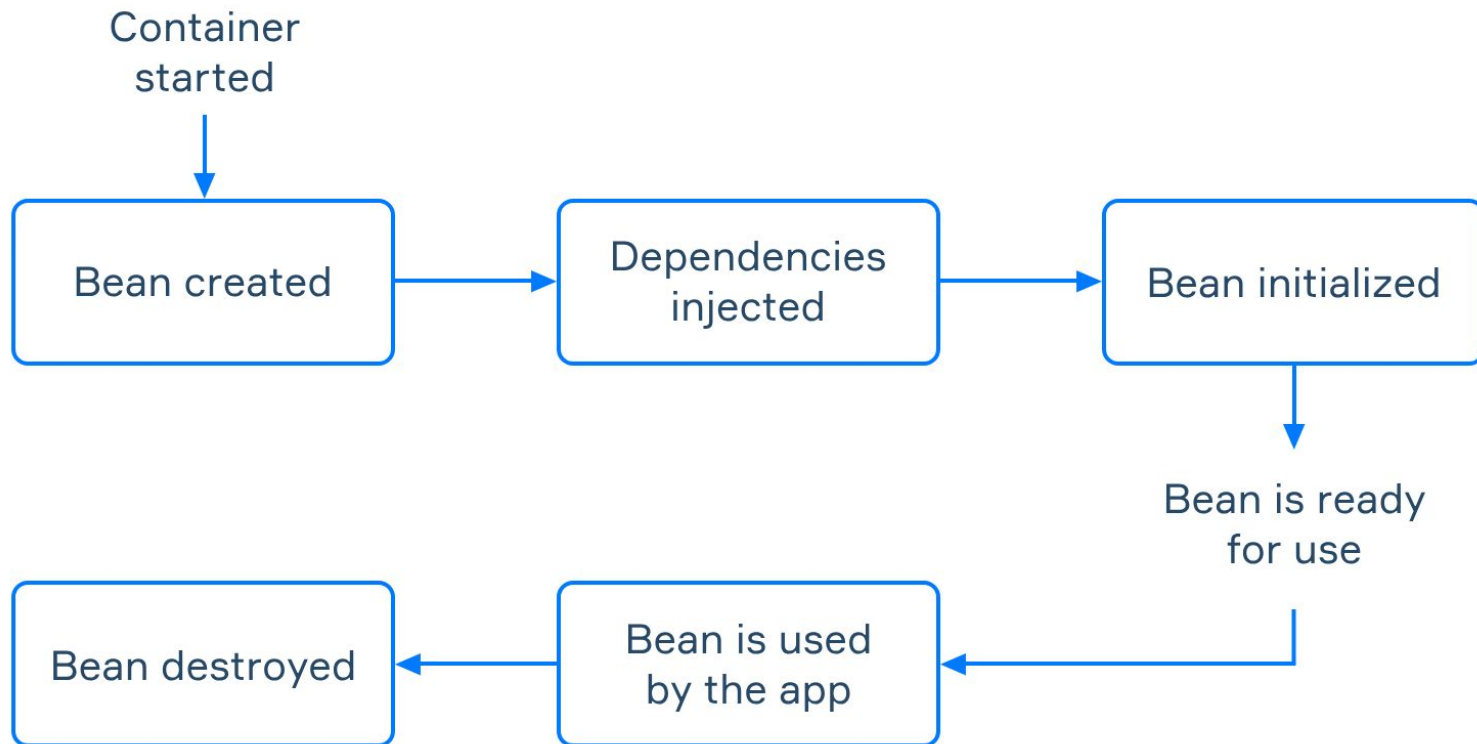
## ► Dependency Injection

- Setter Injection
- Constructor Injection
- Field Injection

## ► Bean Scopes

- **Singleton:** Single instance per Spring IoC container (default).
- **Prototype:** New instance every time requested.
- **Request:** Single instance per HTTP request (web applications).
- **Session:** Single instance per HTTP session (web applications).
- **Global Session:** Single instance per global HTTP session (web applications).

# Lifecycle of the bean – 1000 ft view

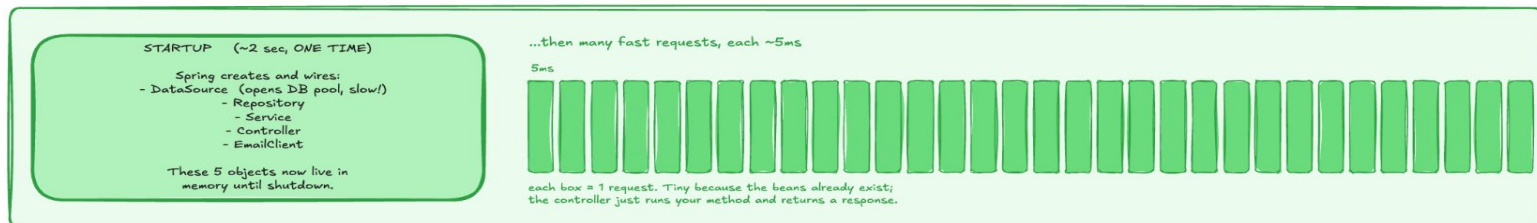


## Why does Spring create beans BEFORE any request arrives?

Some objects (a DB pool, your services, controllers) are **EXPENSIVE** to create — often 100x slower than actually handling a request. So Spring builds them **ONCE** at startup. Imagine the alternative:

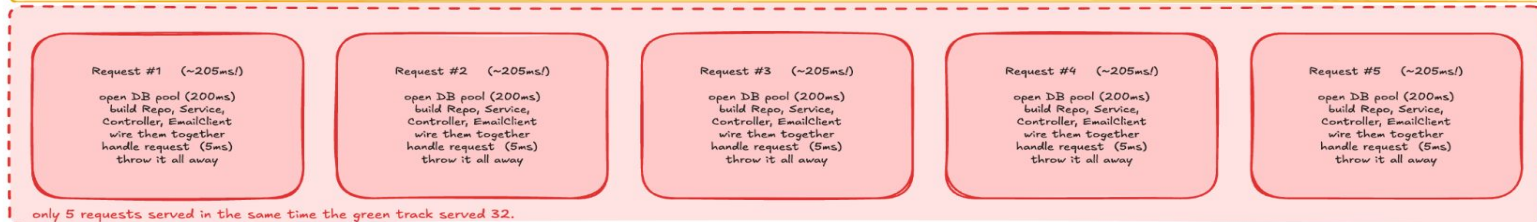
same amount of wall-clock time -->

**WITH SPRING**  
(build beans  
at startup,  
reuse forever)



Same x-axis, same wall-clock time. Only difference: **WHEN** the beans are built.

**NAIVE**  
(rebuild every  
object on each  
request)



### The rule beans are following:

If an object is **EXPENSIVE** to create but **CHEAP** to reuse, build it **ONCE** before requests arrive. DataSource opens real TCP connections to a database. A controller scans annotations. A service may load config. Paying that cost on every request would be insane — so Spring pays it once, at startup, and keeps the objects alive.

### Restaurant analogy:

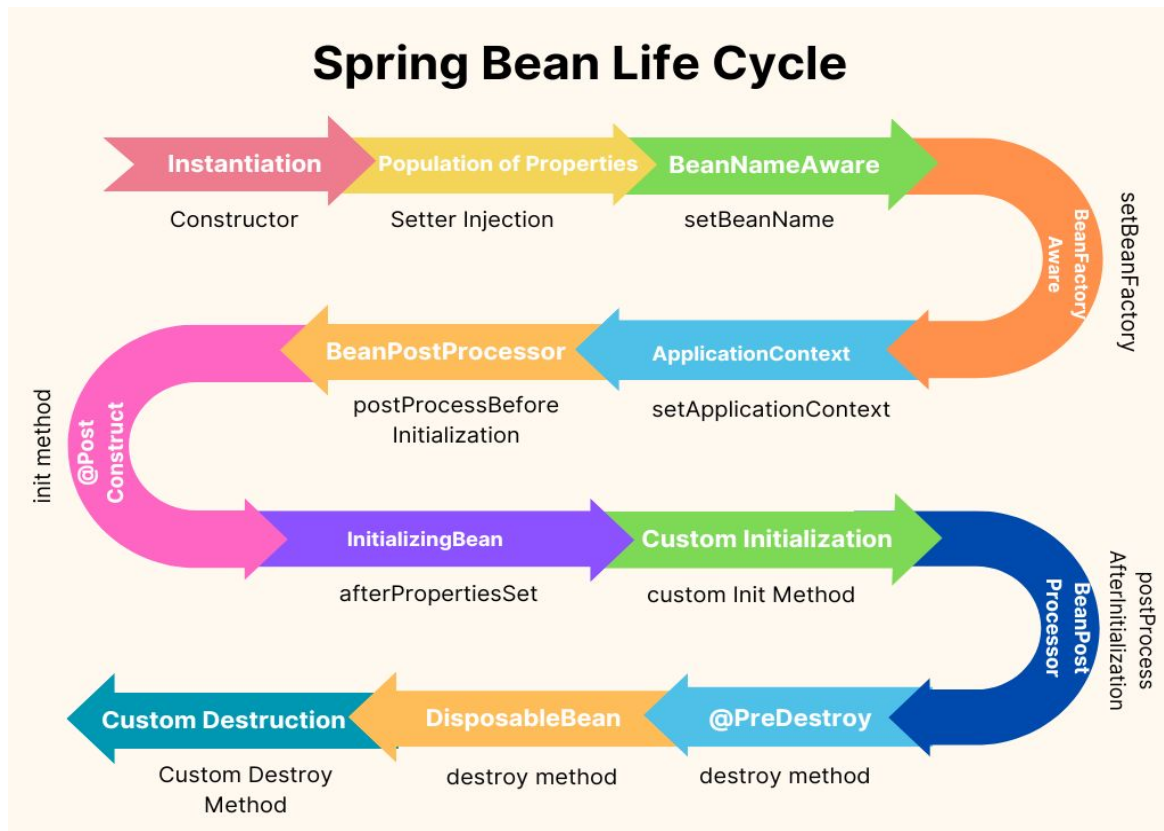
A restaurant does not hire chefs, install ovens, and stock ingredients when a customer walks in. All of that happens **BEFORE** opening hours. Customers arrive -> food is served in minutes, not hours. Spring beans = the restaurant's kitchen, fully set up before the first guest arrives. That is why Spring manages them.

### Why **SPRING** and not you?

Because somebody has to remember: 'this object is built — don't build it again'. With 50+ objects in a real app, tracking that by hand is bug-prone. Spring is that 'somebody': it holds the only reference to each bean and hands it out when asked, so you can never accidentally create a second DataSource or wire things in the wrong order.



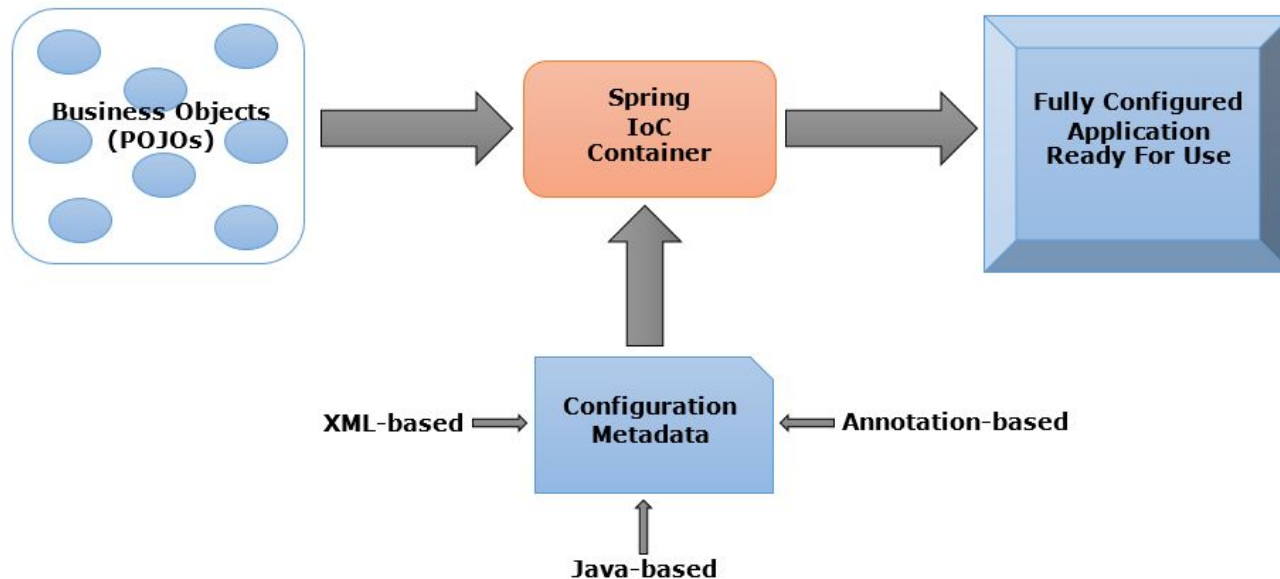
# Life cycle of the bean



# Spring IOC container

## Spring IoC Container

Java Concept Of The Day



# What is spring boot ?

# What is spring boot ?

- ▶ **Spring Boot** is a project from the Spring team that simplifies the process of creating and running Spring applications by providing production-ready defaults and reducing the need for boilerplate code and configuration.
- ▶ **Configuration:** It offers auto-configuration, embedded servers, and a set of opinionated defaults to streamline application setup.

# Features of spring boot

- Auto configuration
- Embedded servers
- Spring boot starters
- Production ready features
- Minimal configurations
- Spring boot cli
- Microservices support

# JAR vs WAR Packaging

- ▶ **JAR (Java Archive):** Used for standalone applications and libraries, includes the application code, dependencies, and an embedded server (for Spring Boot).
- ▶ **WAR (Web Application Archive):** Used for web applications, includes web-specific configurations and resources, and is typically deployed to an external servlet container.

# JAR vs WAR Packaging

## ► Deployment:

- JAR: Self-contained, can be run directly (e.g., `java -jar myapp.jar`).
- WAR: Deployed to a servlet container (e.g., Tomcat, Jetty).

## ► Embedded Server:

- JAR: Includes an embedded server in Spring Boot applications.
- WAR: Requires an external servlet container.

# Creating a web application – Spring

## ► Using Spring:

### 1. Configuration Setup:

- Manually configure `DispatcherServlet` in `web.xml` or use `AbstractAnnotationConfigDispatcherServletInitializer` for Java-based configuration.

### 2. Application Context:

- Define a `WebConfig` class for component scanning and Spring MVC configuration.

### 3. Dependency Management:

- Explicitly add dependencies for Spring MVC and other components in `pom.xml` or `build.gradle`.



# Creating a web application – Spring Boot

► if you have the `spring-boot-starter-web` dependency in your project, Spring

Boot auto-configures everything needed for a web application:

- It sets up a `DispatcherServlet` for handling web requests.
- It auto-configures embedded servers like Tomcat or Jetty.
- It configures view resolvers, message converters, and other components.

# Creating a web application – Spring Boot

## ► Using Spring Boot:

- **Auto-Configuration:**
  - Spring Boot auto-configures `DispatcherServlet` and other necessary components, eliminating the need for manual setup.
- **Simplified Configuration:**
  - Use `@SpringBootApplication` for automatic component scanning and configuration. `application.properties` or `application.yml` simplifies property management.
- **Dependency Management:**
  - Spring Boot starters provide all required dependencies in a single line, streamlining dependency management.

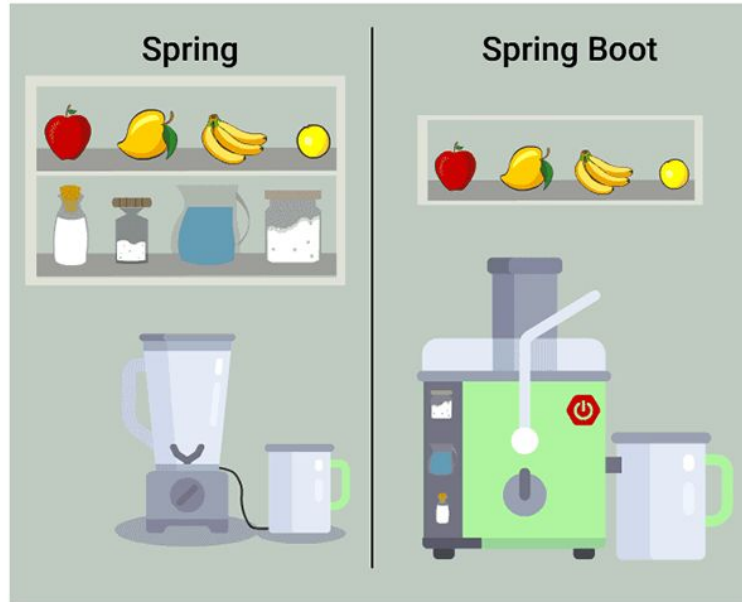
# Spring vs Spring Boot

| Aspect         | Spring                                                                                               | Spring Boot                                                                                           |
|----------------|------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Setup          | Requires manual setup of configuration.                                                              | Simplifies setup with auto-configuration and embedded server.                                         |
| Configuration  | Uses XML or Java-based configuration for setting up the application context and components.          | Uses application.properties or application.yml for configuration. Auto-configures beans and settings. |
| Initialization | Requires explicit configuration of DispatcherServlet and other components in web.xml or Java config. | Automatically configures DispatcherServlet and other necessary components. No need for web.xml.       |
| Dependencies   | Must manually add dependencies in pom.xml or build.gradle.                                           | Spring Boot starters provide a set of dependencies, reducing manual management.                       |

# Spring vs Spring Boot

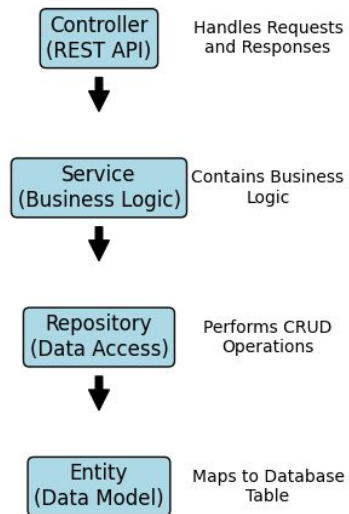
| Aspect               | Spring                                                                 | Spring Boot                                                                                                     |
|----------------------|------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Application Class    | No specific entry point; configuration is spread across various files. | Requires a main application class annotated with <code>@SpringBootApplication</code> to launch the application. |
| Development Speed    | More configuration and setup required; slower setup process.           | Rapid development with minimal configuration; faster setup process.                                             |
| Auto-Configuration   | Requires manual configuration of components and beans.                 | Automatically configures application components based on dependencies and properties.                           |
| Management Endpoints | Requires additional setup for management endpoints (e.g., Actuator).   | Provides built-in management endpoints (e.g., /actuator) for monitoring and management.                         |
| Learning Curve       | Steeper learning curve due to manual configuration and setup.          | Easier to learn with reduced configuration and built-in features.                                               |
| Embedded Server      | Requires separate configuration for embedded servers (e.g., Tomcat).   | Includes an embedded server (e.g., Tomcat, Jetty) by default.                                                   |

# Spring vs Spring Boot

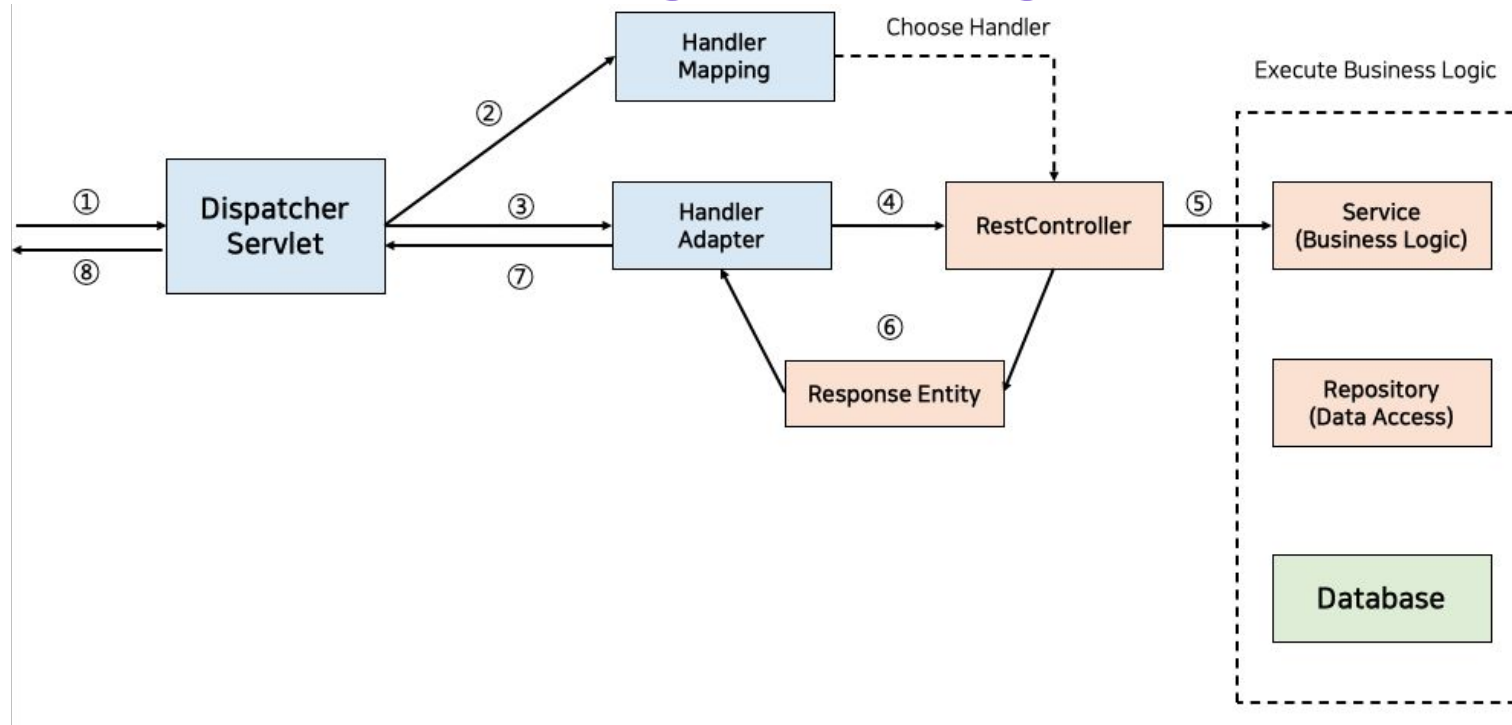


The Difference Between Spring vs. Spring Boot

# Visualizing the web application



# Request Handling in Spring Boot



# Creating Simple controller

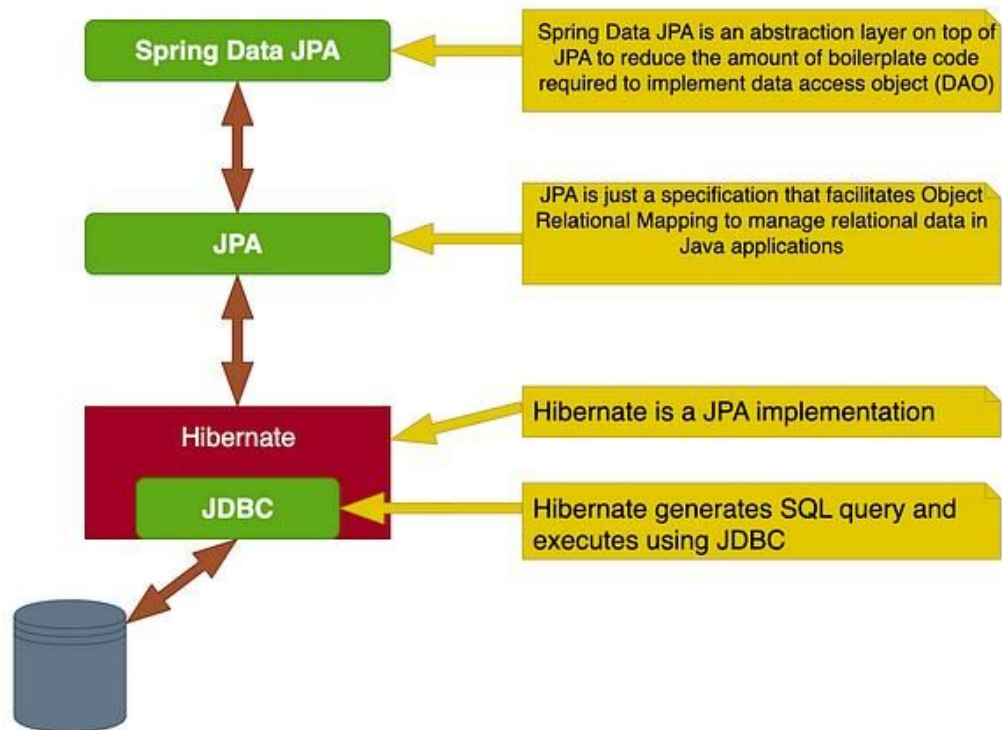


# Creating simple controller

*Let's take look at the code*

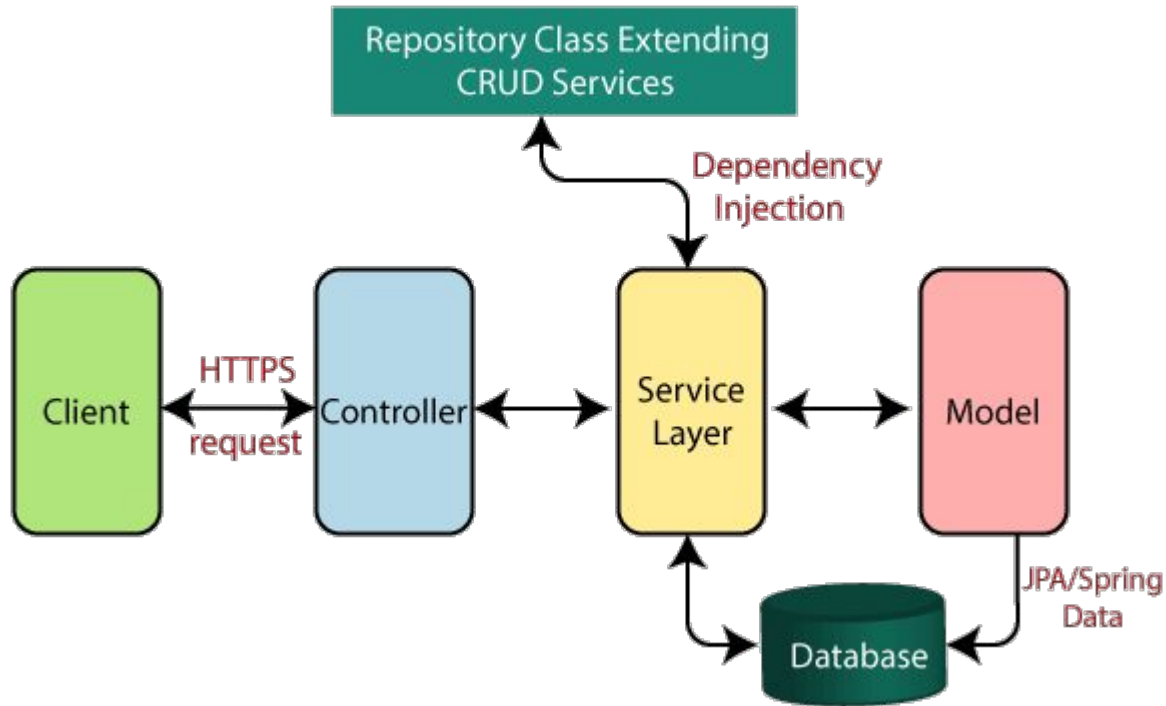
# JPA vs Data JPA vs JDBC vs Hibernate

## Hibernate VS Spring Data JPA



# Spring Boot API flow architecture

## Spring Boot flow architecture



# Enabling actuator

*Spring Boot Actuator provides several built-in endpoints that can be used to monitor and manage your application. some of the commonly used actuator endpoints are present in the next slide*

## *Application.properties*

```
management.endpoints.web.exposure.include=*  
management.endpoint.health.show-details=always
```

# Actuator

| Endpoint                 | Description                       |
|--------------------------|-----------------------------------|
| /actuator/health         | Health status of the application. |
| /actuator/info           | Arbitrary application info.       |
| /actuator/metrics        | Application metrics.              |
| /actuator/env            | Environment properties.           |
| /actuator/configprops    | Configuration properties.         |
| /actuator/beans          | List of all Spring beans.         |
| /actuator/mappings       | All @RequestMapping paths.        |
| /actuator/loggers        | Logger configurations.            |
| /actuator/httptrace      | HTTP trace information.           |
| /actuator/threaddump     | JVM thread dump.                  |
| /actuator/heapdump       | JVM heap dump.                    |
| /actuator/scheduledtasks | Scheduled tasks info.             |
| /actuator/caches         | Cache information.                |

# Best Practices for Modelling Restful endpoints

# Use clear and meaningful resource names

- ▶ Choose descriptive and intuitive names for your resources that accurately represent the entities they represent in your system.

# Use nouns but no verbs in the URI/path

| Purpose                   | Method | Incorrect       | Correct           |
|---------------------------|--------|-----------------|-------------------|
| Retrieves a list of users | GET    | /getAllCars     | /users            |
| Create a new user         | POST   | /createUser     | /users            |
| Delete a user             | DELETE | /deleteUser     | /users/10         |
| Get balance of user       | GET    | /getUserBalance | /users/11/balance |



# Get method should not alter the state

- ▶ Use PUT, POST and DELETE methods instead of GET method to alter the state
- ▶ Do not use GET method or query parameters for state changes:

GET /users/711?activate or

GET /users/711/activate

# Use plural nouns

- ▶ Do not mix up singular and plural nouns. Keep it simple and use only plural nouns for all the resources
- ▶ /cars instead of /car  
/users instead of /user

# Handle errors with http status codes

- ▶ Always return correct http status codes associated with the error handling
- ▶ Returning 200 with an error message is an anti-pattern

Similarly returning 500 with response json is an anti-pattern

# Use sub-resources for relations

- ▶ If a resource is related to another resource use subresources
- ▶ GET /cars/711/drivers (Returns the list of drivers for the car 711)

GET /cars/711/drivers/4 (Returns driver #4 for car 711)

# Provide filtering and sorting operations

- ▶ Allow clients to filter and sort resource collections by providing query parameters
- ▶ For example, **`/products?category=electronics&sort=price`** to fetch electronics products sorted by price.

# Implement Pagination for large resource collections

- ▶ When dealing with resource collections that can become large, provide pagination options to retrieve a subset of the collection at a time. Include query parameters such as page and limit to control the pagination.
- ▶ Along with the paginated results, include metadata in the response payload that indicates the total number of items, current page, and links to navigate to the next and previous pages.

# Don't use unnecessary query parameters

- ▶ Avoid excessive query parameters that don't add significant value to the resource representation or retrieval. Keep the query parameters relevant and focused on filtering, sorting, or pagination.

# Got Questions?



# Thank you